Mod4_Clean-MA

December 3, 2025

[1]:
```
import statsmodels.api as sm
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import sklearn.linear_model as skl
import sklearn.model_selection as skm
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import matplotlib.pyplot as plt
```

[2]:
```
from sklearn.model_selection import train_test_split
from sklearn.model_selection import RandomizedSearchCV
from skorch import NeuralNetClassifier
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.ensemble import \
    (RandomForestRegressor as RF,
     GradientBoostingRegressor as GBR,
     GradientBoostingClassifier as GBC,
     BaggingClassifier,
     RandomForestClassifier)
from scipy.stats import randint
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestRegressor
```

1 1.) Load and Clean

[3]:
```
file_path = r"C:\Users\jstol\Downloads\SDOH_2019_ZIPCODE_1_0 (2).xlsx"
data = pd.read_excel(file_path, sheet_name = 'Data')
data.shape
```

[3]:
```
(41073, 355)
```

[4]:
```
data_MA = pd.DataFrame(data[data['STATE'] == "Massachusetts"])
data = data_MA
```

[6]:
```
columns_to_drop = [
    'STATEFIPS', 'STATE', 'REGION', 'TERRITORY', 'POINT_ZIP', 'YEAR'
]
data = data.drop(columns=columns_to_drop, errors = 'ignore')
data.head()
```

[6]:

	ZIPCODE	ZCTA	ACS_TOT_POP_WT_ZC	ACS_TOT_POP_US_ABOVE1_ZC	\
249	1001	1001.0	17312.0	17133.0	
250	1059	1002.0	30014.0	29866.0	
251	1002	1002.0	30014.0	29866.0	
252	1004	1002.0	30014.0	29866.0	
253	1003	1003.0	11357.0	11357.0	

	ACS_TOT_POP_ABOVE5_ZC	ACS_TOT_POP_ABOVE15_ZC	ACS_TOT_POP_ABOVE16_ZC	\
249	16356.0	14526.0	14386.0	
250	29142.0	26976.0	26656.0	
251	29142.0	26976.0	26656.0	
252	29142.0	26976.0	26656.0	
253	11357.0	11357.0	11357.0	

	ACS_TOT_POP_16_19_ZC	ACS_TOT_POP_ABOVE25_ZC	\
249	405.0	13291.0	
250	3825.0	14069.0	
251	3825.0	14069.0	
252	3825.0	14069.0	
253	6008.0	105.0	

	ACS_TOT_CIVIL_POP_ABOVE18_ZC	…	CDCP_NO_PHY_ACTV_ADULT_C_ZC	\
249	14117.0	…	26.6	
250	26246.0	…	25.8	
251	26246.0	…	25.8	
252	26246.0	…	25.8	
253	11218.0	…	30.0	

	CDCP_PULMONARY_ADULT_C_ZC	POS_DIST_ED_ZP	POS_DIST_MEDSURG_ICU_ZP	\
249	6.9	3.38	3.38	

```
250                      5.0           9.24                     9.24
251                      5.0           7.78                     7.78
252                      5.0           7.05                     7.05
253                      3.3           7.61                     7.61

     POS_DIST_TRAUMA_ZP  POS_DIST_PED_ICU_ZP  POS_DIST_OBSTETRICS_ZP  \
249                3.52                 3.52                    3.38
250               21.06                21.06                    9.24
251               18.13                18.13                    7.78
252               17.89                17.89                    7.05
253               18.96                18.96                    7.61

     POS_DIST_CLINIC_ZP  POS_DIST_ALC_ZP  CEN_AIAN_NH_IND
249                2.76             3.38              0.0
250                2.86            12.85              0.0
251                0.40            13.41              0.0
252                0.51            13.03              0.0
253                0.91            14.07              0.0

[5 rows x 349 columns]
```

2 2.) Presplit Sanitize

[7]:
```
def presplit_sanitize(df, high_missing_thresh = 0.95):
    df = df.copy()
    df.replace([np.inf, -np.inf], np.nan, inplace = True)

    all_nan_cols = df.columns[df.isna().all()]
    if len(all_nan_cols):
        print("Dropping all-NaN cols (global):", list(all_nan_cols))
        df.drop(columns=all_nan_cols, inplace = True)

    miss_ratio = df.isna().mean()
    high_miss_cols = miss_ratio[miss_ratio >= high_missing_thresh].index
    if len(high_miss_cols):
        print(f"Dropping high-missing cols ( {int(high_missing_thresh*100)}%␣
 ↪NaN):", list(high_miss_cols))
        df.drop(columns = high_miss_cols, inplace = True)

    return df

data_clean = presplit_sanitize(data, high_missing_thresh = 0.95)
```

```
Dropping all-NaN cols (global): ['ACS_MEDIAN_HH_INC_NHPI_ZC',
'CCBP_TOT_GAMBLING_ZP']
Dropping high-missing cols ( 95% NaN): ['ACS_MEDIAN_HH_INC_AIAN_ZC',
'CCBP_TOT_CHS_ZP', 'CCBP_TOT_CFS_ZP', 'CCBP_TOT_SFS_ZP', 'CCBP_TOT_EORS_ZP',
```

```
'CCBP_TOT_SHELTERS_ZP']
```

3 3.) Ailment List

[8]:
```
column_list_ailments = [
    'ZIPCODE',
  ␣
 ↪'ACS_PCT_CHILD_DISAB_ZC','ACS_PCT_DISABLE_ZC','ACS_PCT_NONVET_DISABLE_18_64_ZC',
  ␣
 ↪'ACS_PCT_VET_DISABLE_18_64_ZC','CDCP_ARTHRITIS_ADULT_C_ZC','CDCP_ASTHMA_ADULT_C_ZC',
  ␣
 ↪'CDCP_BLOOD_MED_ADULT_C_ZC','CDCP_CHOLES_ADULT_C_ZC','CDCP_CHOLES_SCR_ADULT_C_ZC',
    'CDCP_DOCTOR_VISIT_ADULT_C_ZC','CDCP_KIDNEY_DISEASE_ADULT_C_ZC',
    'CDCP_NO_PHY_ACTV_ADULT_C_ZC','CDCP_PULMONARY_ADULT_C_ZC'
]

data_2 = data_clean[column_list_ailments].copy()
```

4 4.) Standardize Ailment Columns

[9]:
```
id_col = ['ZIPCODE']
cols_to_scale = [col for col in data_2.columns if col not in id_col]

scaler = StandardScaler()
data_norm = data_2.copy()

scaled_values = scaler.fit_transform(data_2[cols_to_scale])

rename_map = {col: f"norm_{col}" for col in cols_to_scale}
for i, col in enumerate(cols_to_scale):
    data_norm[f"norm_{col}"] = scaled_values[:, i]

data_norm = data_norm[id_col + list(rename_map.values())]
print(data_norm.shape)
data_norm.head()
```

```
(681, 14)
```

[9]:

	ZIPCODE	norm_ACS_PCT_CHILD_DISAB_ZC	norm_ACS_PCT_DISABLE_ZC	\
249	1001	0.528513	0.161178	
250	1059	0.007191	-0.514214	
251	1002	0.007191	-0.514214	
252	1004	0.007191	-0.514214	
253	1003	-0.898479	-1.645654	

	norm_ACS_PCT_NONVET_DISABLE_18_64_ZC	norm_ACS_PCT_VET_DISABLE_18_64_ZC	\

249	-0.057325	-0.144760
250	-0.120622	0.425938
251	-0.120622	0.425938
252	-0.120622	0.425938
253	7.236221	NaN

	norm_CDCP_ARTHRITIS_ADULT_C_ZC	norm_CDCP_ASTHMA_ADULT_C_ZC	\
249	1.130504	0.342651	
250	-1.658584	1.051864	
251	-1.658584	1.051864	
252	-1.658584	1.051864	
253	-4.335209	4.395293	

	norm_CDCP_BLOOD_MED_ADULT_C_ZC	norm_CDCP_CHOLES_ADULT_C_ZC	\
249	0.679213	0.488329	
250	-1.628709	-2.704388	
251	-1.628709	-2.704388	
252	-1.628709	-2.704388	
253	-9.462165	-5.645710	

	norm_CDCP_CHOLES_SCR_ADULT_C_ZC	norm_CDCP_DOCTOR_VISIT_ADULT_C_ZC	\
249	0.254560	0.258715	
250	-2.405870	-1.408502	
251	-2.405870	-1.408502	
252	-2.405870	-1.408502	
253	-7.879627	-4.040950	

	norm_CDCP_KIDNEY_DISEASE_ADULT_C_ZC	norm_CDCP_NO_PHY_ACTV_ADULT_C_ZC	\
249	0.841854	0.172928	
250	-0.737605	0.023094	
251	-0.737605	0.023094	
252	-0.737605	0.023094	
253	-2.909361	0.809722	

	norm_CDCP_PULMONARY_ADULT_C_ZC
249	0.756815
250	-0.586063
251	-0.586063
252	-0.586063
253	-1.787585

5 5.) Quartile Buckets

[10]:
```
id_col = ['ZIPCODE']
cols_to_bucket = [c for c in data_norm.columns if c not in id_col]

for c in cols_to_bucket:
    s = pd.to_numeric(data_norm[c], errors = 'coerce')
    q1, q2, q3 = s.quantile([0.25, 0.50, 0.75])
    b = pd.cut(
        s,
        bins = [-np.inf, q1, q2, q3, np.inf],
        labels = [1, 2, 3, 4],
        include_lowest = True
    )
    data_norm[f"bucket_{c}"] = (b.cat.codes.replace(-1, pd.NA) + 1).
 ↪astype('Int64')

print(data_norm.shape)
data_norm.head()
```

```
(681, 27)
```

[10]:

	ZIPCODE	norm_ACS_PCT_CHILD_DISAB_ZC	norm_ACS_PCT_DISABLE_ZC	\
249	1001	0.528513	0.161178	
250	1059	0.007191	-0.514214	
251	1002	0.007191	-0.514214	
252	1004	0.007191	-0.514214	
253	1003	-0.898479	-1.645654	

	norm_ACS_PCT_NONVET_DISABLE_18_64_ZC	norm_ACS_PCT_VET_DISABLE_18_64_ZC	\
249	-0.057325	-0.144760	
250	-0.120622	0.425938	
251	-0.120622	0.425938	
252	-0.120622	0.425938	
253	7.236221	NaN	

	norm_CDCP_ARTHRITIS_ADULT_C_ZC	norm_CDCP_ASTHMA_ADULT_C_ZC	\
249	1.130504	0.342651	
250	-1.658584	1.051864	
251	-1.658584	1.051864	
252	-1.658584	1.051864	
253	-4.335209	4.395293	

	norm_CDCP_BLOOD_MED_ADULT_C_ZC	norm_CDCP_CHOLES_ADULT_C_ZC	\
249	0.679213	0.488329	
250	-1.628709	-2.704388	
251	-1.628709	-2.704388	

```
252                      -1.628709                    -2.704388
253                      -9.462165                    -5.645710

     norm_CDCP_CHOLES_SCR_ADULT_C_ZC  …  \
249                         0.254560  …
250                        -2.405870  …
251                        -2.405870  …
252                        -2.405870  …
253                        -7.879627  …

     bucket_norm_ACS_PCT_VET_DISABLE_18_64_ZC  \
249                                         3
250                                         4
251                                         4
252                                         4
253                                      <NA>

     bucket_norm_CDCP_ARTHRITIS_ADULT_C_ZC  \
249                                      4
250                                      1
251                                      1
252                                      1
253                                      1

     bucket_norm_CDCP_ASTHMA_ADULT_C_ZC  \
249                                   3
250                                   4
251                                   4
252                                   4
253                                   4

     bucket_norm_CDCP_BLOOD_MED_ADULT_C_ZC  \
249                                      4
250                                      1
251                                      1
252                                      1
253                                      1

     bucket_norm_CDCP_CHOLES_ADULT_C_ZC  \
249                                   3
250                                   1
251                                   1
252                                   1
253                                   1

     bucket_norm_CDCP_CHOLES_SCR_ADULT_C_ZC  \
249                                      3
```

```
250                                          1
251                                          1
252                                          1
253                                          1

     bucket_norm_CDCP_DOCTOR_VISIT_ADULT_C_ZC  \
249                                          3
250                                          1
251                                          1
252                                          1
253                                          1

     bucket_norm_CDCP_KIDNEY_DISEASE_ADULT_C_ZC  \
249                                            4
250                                            1
251                                            1
252                                            1
253                                            1

     bucket_norm_CDCP_NO_PHY_ACTV_ADULT_C_ZC  \
249                                          3
250                                          3
251                                          3
252                                          3
253                                          4

     bucket_norm_CDCP_PULMONARY_ADULT_C_ZC
249                                       4
250                                       2
251                                       2
252                                       2
253                                       1

[5 rows x 27 columns]
```

[11]:
```
cols_to_sum = [c for c in data_norm.columns if c.startswith("bucket_")]
data_norm["zip_bucket_score"] = data_norm[cols_to_sum].sum(axis=1, min_count=1)
data_norm_bucket = data_norm[['ZIPCODE', 'zip_bucket_score']]
data_norm_bucket.head()
```

[11]:
```
     ZIPCODE  zip_bucket_score
249     1001                44
250     1059                27
251     1002                27
252     1004                27
253     1003                21
```

[12]:
```
data_norm_bucket[data_norm_bucket['zip_bucket_score'] ==␣
↪data_norm_bucket['zip_bucket_score'].max()]
```

[12]:

	ZIPCODE	zip_bucket_score
878	2664	49

[13]:
```
data_norm_bucket[data_norm_bucket['zip_bucket_score'] ==␣
↪data_norm_bucket['zip_bucket_score'].min()]
```

[13]:

	ZIPCODE	zip_bucket_score
735	2203	4
736	2201	4

6 6.) Merge Score and Ailments

[14]:
```
data_merged = data_clean.merge(
    data_norm_bucket[['ZIPCODE', 'zip_bucket_score']],
    on = 'ZIPCODE',
    how = 'left'
)

cols_to_drop = [
   ␣
↪'ACS_PCT_CHILD_DISAB_ZC','ACS_PCT_DISABLE_ZC','ACS_PCT_NONVET_DISABLE_18_64_ZC',
   ␣
↪'ACS_PCT_VET_DISABLE_18_64_ZC','CDCP_ARTHRITIS_ADULT_C_ZC','CDCP_ASTHMA_ADULT_C_ZC',
   ␣
↪'CDCP_BLOOD_MED_ADULT_C_ZC','CDCP_CHOLES_ADULT_C_ZC','CDCP_CHOLES_SCR_ADULT_C_ZC',
    'CDCP_DOCTOR_VISIT_ADULT_C_ZC','CDCP_KIDNEY_DISEASE_ADULT_C_ZC',
    'CDCP_NO_PHY_ACTV_ADULT_C_ZC','CDCP_PULMONARY_ADULT_C_ZC'
]
data_bucket_score = data_merged.drop(columns = cols_to_drop, errors = 'ignore')
data_bucket_score.head()
```

[14]:

	ZIPCODE	ZCTA	ACS_TOT_POP_WT_ZC	ACS_TOT_POP_US_ABOVE1_ZC	\
0	1001	1001.0	17312.0	17133.0	
1	1059	1002.0	30014.0	29866.0	
2	1002	1002.0	30014.0	29866.0	
3	1004	1002.0	30014.0	29866.0	
4	1003	1003.0	11357.0	11357.0	

	ACS_TOT_POP_ABOVE5_ZC	ACS_TOT_POP_ABOVE15_ZC	ACS_TOT_POP_ABOVE16_ZC	\
0	16356.0	14526.0	14386.0	
1	29142.0	26976.0	26656.0	
2	29142.0	26976.0	26656.0	
3	29142.0	26976.0	26656.0	

```
4                11357.0                 11357.0                 11357.0

   ACS_TOT_POP_16_19_ZC  ACS_TOT_POP_ABOVE25_ZC  ACS_TOT_CIVIL_POP_ABOVE18_ZC  \
0                 405.0                 13291.0                       14117.0
1                3825.0                 14069.0                       26246.0
2                3825.0                 14069.0                       26246.0
3                3825.0                 14069.0                       26246.0
4                6008.0                   105.0                       11218.0

   …  ERS_RUCA2_2010_ZP  POS_DIST_ED_ZP  POS_DIST_MEDSURG_ICU_ZP  \
0  …                1.0            3.38                     3.38
1  …                1.0            9.24                     9.24
2  …                1.0            7.78                     7.78
3  …                1.0            7.05                     7.05
4  …                1.0            7.61                     7.61

   POS_DIST_TRAUMA_ZP  POS_DIST_PED_ICU_ZP  POS_DIST_OBSTETRICS_ZP  \
0                3.52                 3.52                    3.38
1               21.06                21.06                    9.24
2               18.13                18.13                    7.78
3               17.89                17.89                    7.05
4               18.96                18.96                    7.61

   POS_DIST_CLINIC_ZP  POS_DIST_ALC_ZP  CEN_AIAN_NH_IND  zip_bucket_score
0                2.76             3.38              0.0                44
1                2.86            12.85              0.0                27
2                0.40            13.41              0.0                27
3                0.51            13.03              0.0                27
4                0.91            14.07              0.0                21

[5 rows x 329 columns]
```

7 7.) PCA

[15]:
```
ailment_cols = [
    ␣
  ↪'ACS_PCT_CHILD_DISAB_ZC','ACS_PCT_DISABLE_ZC','ACS_PCT_NONVET_DISABLE_18_64_ZC',
    ␣
  ↪'ACS_PCT_VET_DISABLE_18_64_ZC','CDCP_ARTHRITIS_ADULT_C_ZC','CDCP_ASTHMA_ADULT_C_ZC',
    ␣
  ↪'CDCP_BLOOD_MED_ADULT_C_ZC','CDCP_CHOLES_ADULT_C_ZC','CDCP_CHOLES_SCR_ADULT_C_ZC',
     'CDCP_DOCTOR_VISIT_ADULT_C_ZC','CDCP_KIDNEY_DISEASE_ADULT_C_ZC',
     'CDCP_NO_PHY_ACTV_ADULT_C_ZC','CDCP_PULMONARY_ADULT_C_ZC'
]
ailment_cols = [c for c in ailment_cols if c in data_clean.columns]
```

```
X = data_clean[ailment_cols].apply(pd.to_numeric, errors='coerce')

pipe = Pipeline(steps = [
    ('imputer', SimpleImputer(strategy = 'median')),
    ('scaler', StandardScaler()),
    ('pca', PCA(n_components = 3, random_state = 77))
])

pcs = pipe.fit_transform(X)
pca_df = pd.DataFrame(pcs, columns = ['PC1','PC2','PC3'], index = X.index)

data_pca = data_clean.copy()
data_pca[['PC1','PC2','PC3']] = np.nan
data_pca.loc[X.index, ['PC1','PC2','PC3']] = pca_df[['PC1','PC2','PC3']]

#!!!
expl_var = pipe.named_steps['pca'].explained_variance_ratio_
print("Explained variance by PC:", expl_var, " | Cumulative:", expl_var.
 ↪cumsum())

loadings = pd.DataFrame(
    pipe.named_steps['pca'].components_.T,
    index = X.columns,
    columns = ['PC1','PC2','PC3']
).sort_values('PC1', key = lambda s: s.abs(), ascending=False)
print(loadings.head(10))
```

```
Explained variance by PC: [0.38012388 0.32629698 0.07864255]  | Cumulative:
[0.38012388 0.70642087 0.78506342]
                                       PC1       PC2       PC3
CDCP_ARTHRITIS_ADULT_C_ZC        -0.437939 -0.022075  0.007706
CDCP_CHOLES_ADULT_C_ZC           -0.416040 -0.133082 -0.040975
CDCP_BLOOD_MED_ADULT_C_ZC        -0.406295 -0.126467 -0.099342
CDCP_KIDNEY_DISEASE_ADULT_C_ZC   -0.361484  0.231098 -0.137855
CDCP_DOCTOR_VISIT_ADULT_C_ZC     -0.328798 -0.267033  0.030628
CDCP_PULMONARY_ADULT_C_ZC        -0.328261  0.288871 -0.060932
CDCP_CHOLES_SCR_ADULT_C_ZC       -0.269302 -0.362600  0.091507
ACS_PCT_DISABLE_ZC               -0.168036  0.257006  0.191144
CDCP_NO_PHY_ACTV_ADULT_C_ZC      -0.113994  0.426303 -0.162535
ACS_PCT_NONVET_DISABLE_18_64_ZC  -0.070431  0.359671  0.229193
```

8 8.) Merge PC's with Bucket Score

[16]:
```
data_pca_buck = data_pca.merge(
    data_bucket_score[['ZIPCODE', 'zip_bucket_score']],
    on = 'ZIPCODE',
    how = 'left'
```

```
)
data_pca_buck[['PC1','PC2','PC3','zip_bucket_score']].corr()

data_pca_buck = data_pca_buck.drop(columns = ailment_cols, errors = 'ignore')
data_pca_buck.head()
```

[16]:

	ZIPCODE	ZCTA	ACS_TOT_POP_WT_ZC	ACS_TOT_POP_US_ABOVE1_ZC	\
0	1001	1001.0	17312.0	17133.0	
1	1059	1002.0	30014.0	29866.0	
2	1002	1002.0	30014.0	29866.0	
3	1004	1002.0	30014.0	29866.0	
4	1003	1003.0	11357.0	11357.0	

	ACS_TOT_POP_ABOVE5_ZC	ACS_TOT_POP_ABOVE15_ZC	ACS_TOT_POP_ABOVE16_ZC	\
0	16356.0	14526.0	14386.0	
1	29142.0	26976.0	26656.0	
2	29142.0	26976.0	26656.0	
3	29142.0	26976.0	26656.0	
4	11357.0	11357.0	11357.0	

	ACS_TOT_POP_16_19_ZC	ACS_TOT_POP_ABOVE25_ZC	ACS_TOT_CIVIL_POP_ABOVE18_ZC	\
0	405.0	13291.0	14117.0	
1	3825.0	14069.0	26246.0	
2	3825.0	14069.0	26246.0	
3	3825.0	14069.0	26246.0	
4	6008.0	105.0	11218.0	

	…	POS_DIST_TRAUMA_ZP	POS_DIST_PED_ICU_ZP	POS_DIST_OBSTETRICS_ZP	\
0	…	3.52	3.52	3.38	
1	…	21.06	21.06	9.24	
2	…	18.13	18.13	7.78	
3	…	17.89	17.89	7.05	
4	…	18.96	18.96	7.61	

	POS_DIST_CLINIC_ZP	POS_DIST_ALC_ZP	CEN_AIAN_NH_IND	PC1	PC2	\
0	2.76	3.38	0.0	-1.762600	0.411885	
1	2.86	12.85	0.0	4.202610	1.860238	
2	0.40	13.41	0.0	4.202610	1.860238	
3	0.51	13.03	0.0	4.202610	1.860238	
4	0.91	14.07	0.0	12.938207	8.911970	

	PC3	zip_bucket_score
0	0.185884	44
1	-0.295629	27
2	-0.295629	27
3	-0.295629	27
4	1.708331	21

[5 rows x 332 columns]

[17]:
```
data_pca_buck[['PC1','PC2','PC3','zip_bucket_score']].corr()
```

[17]:

	PC1	PC2	PC3	zip_bucket_score
PC1	1.000000e+00	1.219227e-16	-1.749467e-16	-0.760995
PC2	1.219227e-16	1.000000e+00	1.784922e-16	0.403727
PC3	-1.749467e-16	1.784922e-16	1.000000e+00	-0.036933
zip_bucket_score	-7.609946e-01	4.037267e-01	-3.693256e-02	1.000000

9 9.) Zip_Bucket_Score Outcome Model

[18]:
```
drop_columns = ['zip_bucket_score', 'PC1', 'PC2', 'PC3']
X = data_pca_buck.drop(drop_columns, axis = 1, errors = 'ignore')
Y = data_pca_buck['zip_bucket_score']

X = X.drop('ZIPCODE', axis = 1, errors = 'ignore')

mask_target = Y.notna()
X = X.loc[mask_target]
Y = Y.loc[mask_target]

n = X.shape[0]
test_df = X.sample(n = int(n * 0.2), random_state = 77)
x_test_idx = test_df.index

x_test = X.loc[x_test_idx]
y_test = Y.loc[x_test_idx]

x_train = X.drop(x_test_idx)
y_train = Y.drop(x_test_idx)

x_train = pd.get_dummies(x_train, drop_first = True).astype(float)
x_test  = pd.get_dummies(x_test,  drop_first = True).astype(float)
x_test  = x_test.reindex(columns = x_train.columns, fill_value = 0)

imputer = SimpleImputer(strategy = "median")
x_train_imp_df = pd.DataFrame(imputer.fit_transform(x_train), columns = x_train.
 ↪columns, index = x_train.index)
x_test_imp_df  = pd.DataFrame(imputer.transform(x_test),  columns = x_test.
 ↪columns,  index = x_test.index)
```

10 10.) Random Forest Fit and Predictions

[19]:
```
rf = RandomForestRegressor(n_estimators = 600, random_state = 77, n_jobs = -1)
rf.fit(x_train_imp_df, y_train)

preds = rf.predict(x_test_imp_df)
mae  = mean_absolute_error(y_test, preds)
rmse = mean_squared_error(y_test, preds, squared = False)
r2   = r2_score(y_test, preds)

print(f"RandomForest MAE: {mae:.4f}")
print(f"RandomForest RMSE: {rmse:.4f}")
print(f"RandomForest R²: {r2:.4f}")
```

```
RandomForest MAE: 2.1069
RandomForest RMSE: 3.0763
RandomForest R²: 0.8545
```

```
C:\Users\jstol\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:483:
FutureWarning: 'squared' is deprecated in version 1.4 and will be removed in
1.6. To calculate the root mean squared error, use the
function'root_mean_squared_error'.
  warnings.warn(
```

[20]:
```
important_params = {
    "n_estimators": rf.n_estimators,
    "max_depth": rf.max_depth,
    "max_features": rf.max_features,
    "min_samples_split": rf.min_samples_split,
    "min_samples_leaf": rf.min_samples_leaf,
    "bootstrap": rf.bootstrap
}

print(important_params)
```

```
{'n_estimators': 600, 'max_depth': None, 'max_features': 1.0,
'min_samples_split': 2, 'min_samples_leaf': 1, 'bootstrap': True}
```

11 11.) Top Features

[21]:
```
fi = pd.DataFrame(
    {'feature': x_train_imp_df.columns, 'importance': rf.feature_importances_}
).sort_values('importance', ascending = False)

print("\nTop 10 Most Important Features:")
print(fi.head(10).to_string(index = False))
```

```
Top 10 Most Important Features:
                      feature  importance
  ACS_MEDIAN_HH_INC_WHITE_ZC    0.227635
           ACS_MEDIAN_AGE_ZC    0.059009
      ACS_PCT_HH_INC_49999_ZC    0.057525
     ACS_PCT_HH_SMARTPHONE_ZC    0.052032
      ACS_PCT_PRIVATE_EMPL_ZC    0.038879
       ACS_PCT_AGE_ABOVE65_ZC    0.030782
         ACS_PCT_AGE_18_44_ZC    0.027885
       ACS_PCT_PRIVATE_ANY_ZC    0.026907
         ACS_PCT_OTHER_INS_ZC    0.023764
ACS_PCT_HOUSEHOLDER_ASIAN_ZC    0.021264
```

The model is repeatedly pulling information from different correlated variables that describe the same underlying concept. Thankfully Random Forests don't care about multicollinearity, because we are seeing: - income twice - age three times - insurance three times - socioeconomic proxies multiple times

12 12.) PC1 Outcome Random Forest Regression Model

[22]:
```
drop_columns = ['zip_bucket_score', 'PC1', 'PC2', 'PC3']
X_2 = data_pca_buck.drop(drop_columns, axis = 1, errors = 'ignore')

Y_2 = data_pca_buck['PC1']


X_2 = X_2.drop('ZIPCODE', axis = 1, errors = 'ignore')

mask_target = Y_2.notna()
X_2 = X_2.loc[mask_target]
Y_2 = Y_2.loc[mask_target]

n = X_2.shape[0]
test_df = X_2.sample(n = int(n * 0.20), random_state=77)
x_2_test_idx = test_df.index

x_2_test = X_2.loc[x_2_test_idx]
y_2_test = Y_2.loc[x_2_test_idx]

x_2_train = X_2.drop(x_2_test_idx)
y_2_train = Y_2.drop(x_2_test_idx)


x_2_train = pd.get_dummies(x_2_train, drop_first = True).astype(float)
x_2_test  = pd.get_dummies(x_2_test,  drop_first = True).astype(float)

x_2_test = x_2_test.reindex(columns = x_2_train.columns, fill_value=0)
```

```
imputer = SimpleImputer(strategy = "median")

x_2_train_imp = pd.DataFrame(
    imputer.fit_transform(x_2_train),
    columns = x_2_train.columns,
    index = x_2_train.index
)

x_2_test_imp = pd.DataFrame(
    imputer.transform(x_2_test),
    columns = x_2_test.columns,
    index = x_2_test.index
)
```

[23]:
```
rf_2 = RandomForestRegressor(
    n_estimators = 600,
    random_state = 77,
    n_jobs = -1
)

rf_2.fit(x_2_train_imp, y_2_train)

preds_2 = rf_2.predict(x_2_test_imp)

mae_2  = mean_absolute_error(y_2_test, preds_2)
rmse_2 = mean_squared_error(y_2_test, preds_2, squared = False)
r2_2   = r2_score(y_2_test, preds_2)

print(f"RandomForest (PC1) MAE:  {mae_2:.4f}")
print(f"RandomForest (PC1) RMSE: {rmse_2:.4f}")
print(f"RandomForest (PC1) R²:   {r2_2:.4f}")
```

```
RandomForest (PC1) MAE:  0.5938
RandomForest (PC1) RMSE: 1.2669
RandomForest (PC1) R²:   0.5992

C:\Users\jstol\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:483:
FutureWarning: 'squared' is deprecated in version 1.4 and will be removed in
1.6. To calculate the root mean squared error, use the
function'root_mean_squared_error'.
  warnings.warn(
```

[24]:
```
fi_2 = pd.DataFrame(
    {
        'feature': x_2_train_imp.columns,
        'importance': rf_2.feature_importances_
```

```
    }
).sort_values('importance', ascending = False)

print("\nTop 10 Most Important Features (PC1 Model):")
print(fi_2.head(10).to_string(index = False))
```

```
Top 10 Most Important Features (PC1 Model):
                   feature  importance
      ACS_PCT_AGE_18_44_ZC    0.507096
    ACS_PCT_AGE_ABOVE65_ZC    0.146400
     ACS_PCT_HH_ABOVE65_ZC    0.025862
ACS_PCT_HH_ALONE_ABOVE65_ZC    0.022154
      ACS_PCT_NOT_LABOR_ZC    0.019018
      ACS_PCT_AGE_50_64_ZC    0.013753
      ACS_PCT_AGE_45_64_ZC    0.013489
    ACS_PCT_AGE_ABOVE80_ZC    0.013142
  ACS_MEDIAN_AGE_FEMALE_ZC    0.007567
      ACS_PCT_VACANT_HU_ZC    0.007248
```

13 13.) PC2 Outcome Random Forest Regression Model

[25]:
```
drop_columns = ['zip_bucket_score', 'PC1', 'PC2', 'PC3']
X_3 = data_pca_buck.drop(drop_columns, axis = 1, errors = 'ignore')

Y_3 = data_pca_buck['PC2']

X_3 = X_3.drop('ZIPCODE', axis = 1, errors = 'ignore')

mask_target_3 = Y_3.notna()
X_3 = X_3.loc[mask_target_3]
Y_3 = Y_3.loc[mask_target_3]


n3 = X_3.shape[0]
test_df_3 = X_3.sample(n = int(n3 * 0.20), random_state = 77)
x_3_test_idx = test_df_3.index

x_3_test = X_3.loc[x_3_test_idx]
y_3_test = Y_3.loc[x_3_test_idx]

x_3_train = X_3.drop(x_3_test_idx)
y_3_train = Y_3.drop(x_3_test_idx)


x_3_train = pd.get_dummies(x_3_train, drop_first = True).astype(float)
x_3_test  = pd.get_dummies(x_3_test,  drop_first = True).astype(float)
```

```
x_3_test = x_3_test.reindex(columns = x_3_train.columns, fill_value = 0)


imputer_3 = SimpleImputer(strategy="median")

x_3_train_imp = pd.DataFrame(
    imputer_3.fit_transform(x_3_train),
    columns = x_3_train.columns,
    index = x_3_train.index
)

x_3_test_imp = pd.DataFrame(
    imputer_3.transform(x_3_test),
    columns = x_3_test.columns,
    index = x_3_test.index
)
```

[26]:
```
rf_3 = RandomForestRegressor(
    n_estimators = 600,
    random_state = 77,
    n_jobs = -1
)

rf_3.fit(x_3_train_imp, y_3_train)

preds_3 = rf_3.predict(x_3_test_imp)

mae_3  = mean_absolute_error(y_3_test, preds_3)
rmse_3 = mean_squared_error(y_3_test, preds_3, squared = False)
r2_3   = r2_score(y_3_test, preds_3)

print(f"RandomForest (PC2) MAE:  {mae_3:.4f}")
print(f"RandomForest (PC2) RMSE: {rmse_3:.4f}")
print(f"RandomForest (PC2) R²:   {r2_3:.4f}")
```

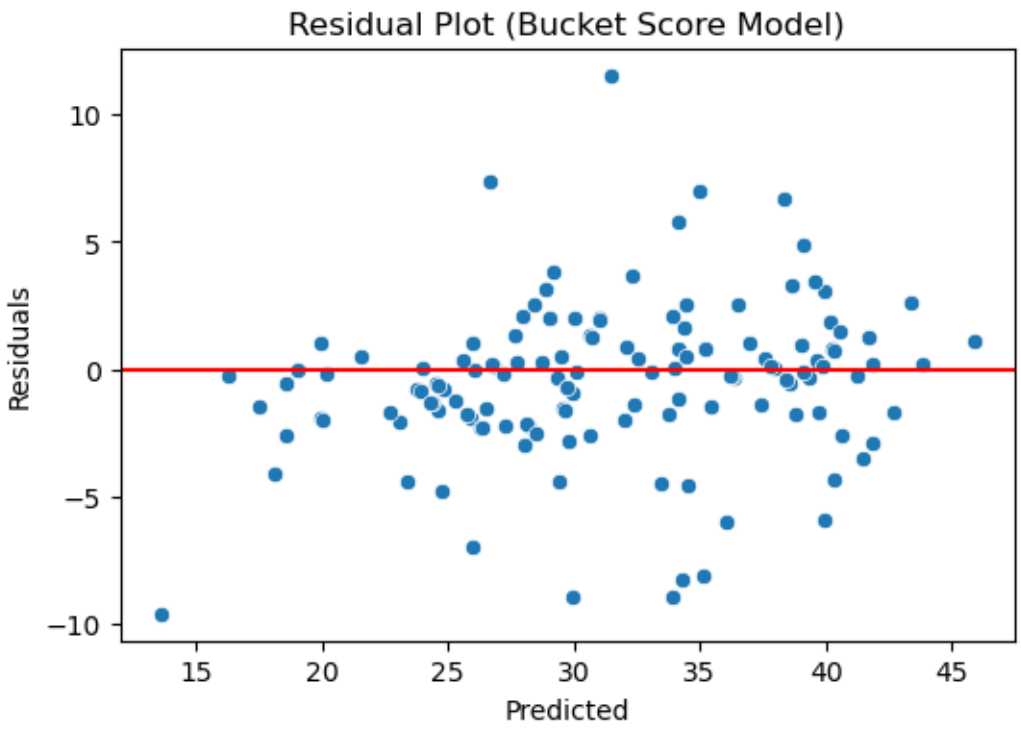
```
RandomForest (PC2) MAE:  0.3607
RandomForest (PC2) RMSE: 0.5833
RandomForest (PC2) R²:   0.9229

C:\Users\jstol\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:483:
FutureWarning: 'squared' is deprecated in version 1.4 and will be removed in
1.6. To calculate the root mean squared error, use the
function'root_mean_squared_error'.
  warnings.warn(
```

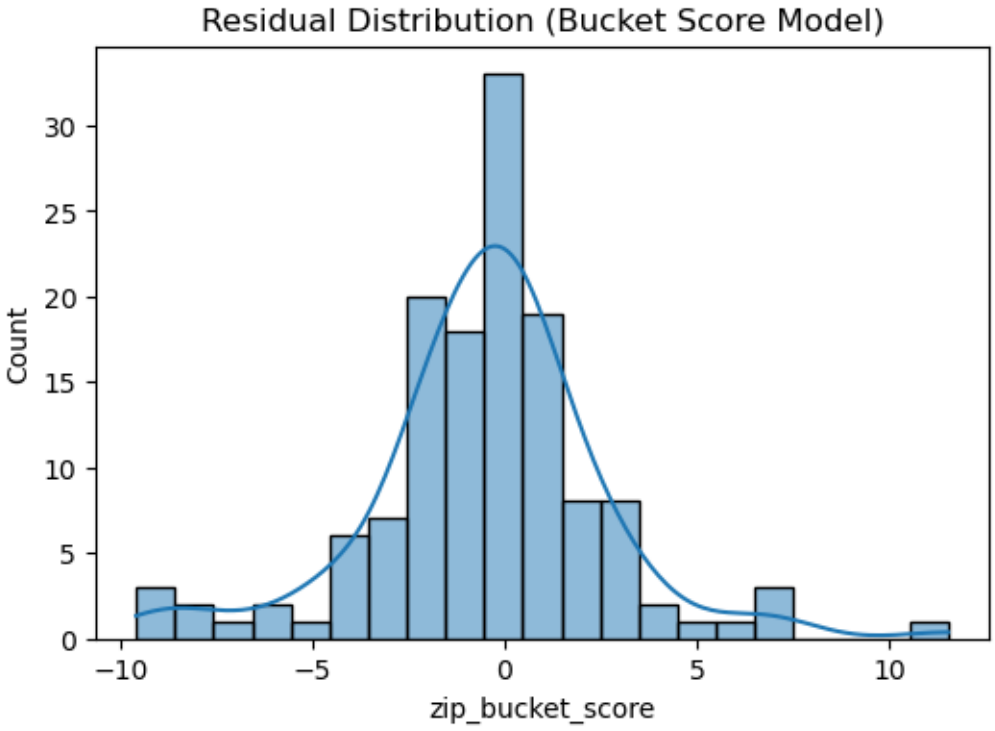
[27]:
```
fi_3 = pd.DataFrame(
    {
```

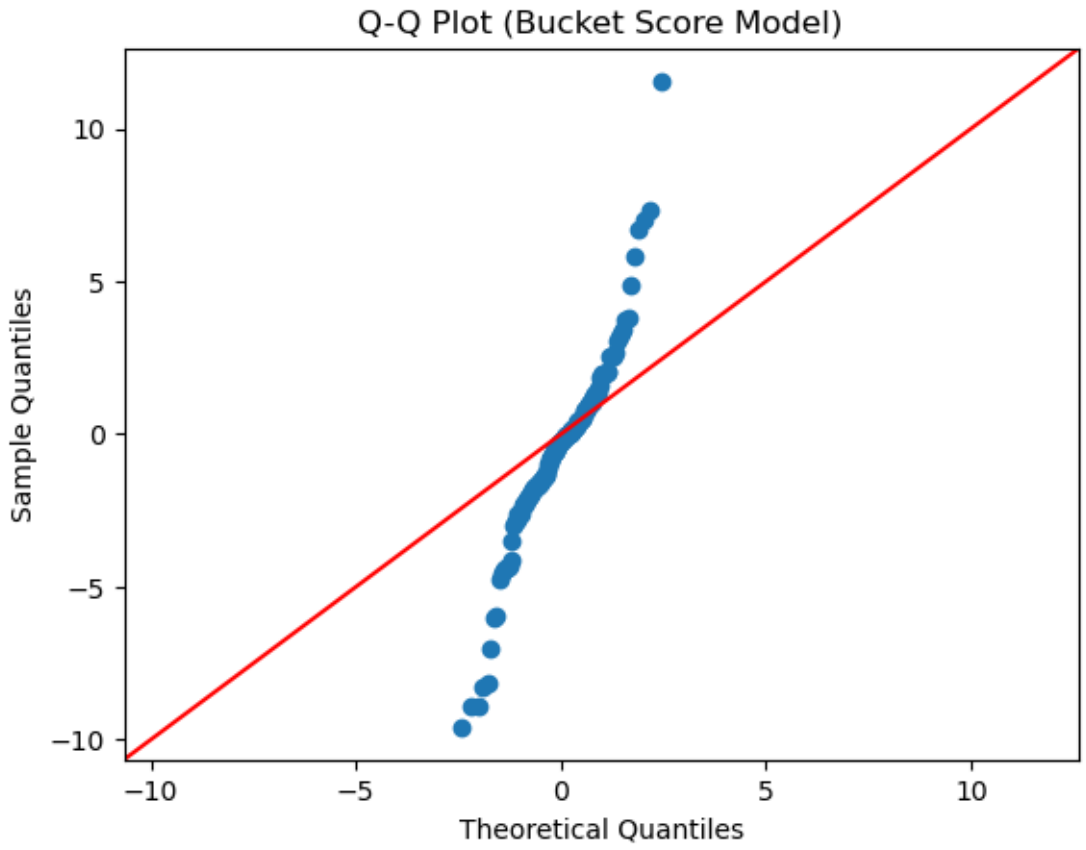
```
        'feature': x_3_train_imp.columns,
        'importance': rf_3.feature_importances_
    }
).sort_values('importance', ascending = False)

print("\nTop 10 Most Important Features (PC2 Model):")
print(fi_3.head(10).to_string(index = False))
```

```
Top 10 Most Important Features (PC2 Model):
                      feature  importance
          ACS_PER_CAPITA_INC_ZC    0.284702
 ACS_PCT_PERSON_INC_ABOVE200_ZC    0.203353
           ACS_MEDIAN_HH_INC_ZC    0.057264
       ACS_PCT_HH_PUB_ASSIST_ZC    0.056266
ACS_PCT_HH_FOOD_STMP_BLW_POV_ZC    0.037932
        ACS_PCT_HH_FOOD_STMP_ZC    0.035271
            ACS_MEDIAN_INC_M_ZC    0.028120
 ACS_PCT_HEALTH_INC_BELOW137_ZC    0.025888
 ACS_PCT_HEALTH_INC_ABOVE400_ZC    0.025880
       ACS_MEDIAN_NONVET_INC_ZC    0.023930
```

[33]:
```
# Residual diagnostics for Bucket Score model
residuals = y_test - preds

plt.figure(figsize=(6,4))
sns.scatterplot(x=preds, y=residuals)
plt.axhline(0, color='red')
plt.title("Residual Plot (Bucket Score Model)")
plt.xlabel("Predicted")
plt.ylabel("Residuals")
plt.show()

plt.figure(figsize=(6,4))
sns.histplot(residuals, kde=True)
plt.title("Residual Distribution (Bucket Score Model)")
plt.show()

import scipy.stats as stats
sm.qqplot(residuals, line='45')
plt.title("Q-Q Plot (Bucket Score Model)")
plt.show()
```


Residual Plot (Bucket Score Model)
Residuals
Predicted
10
5
0
−5
−10
15
20
25
30
35
40
45


Residual Distribution (Bucket Score Model)
Count
zip_bucket_score
30
25
20
15
10
5
0
−10
−5
0
5
10

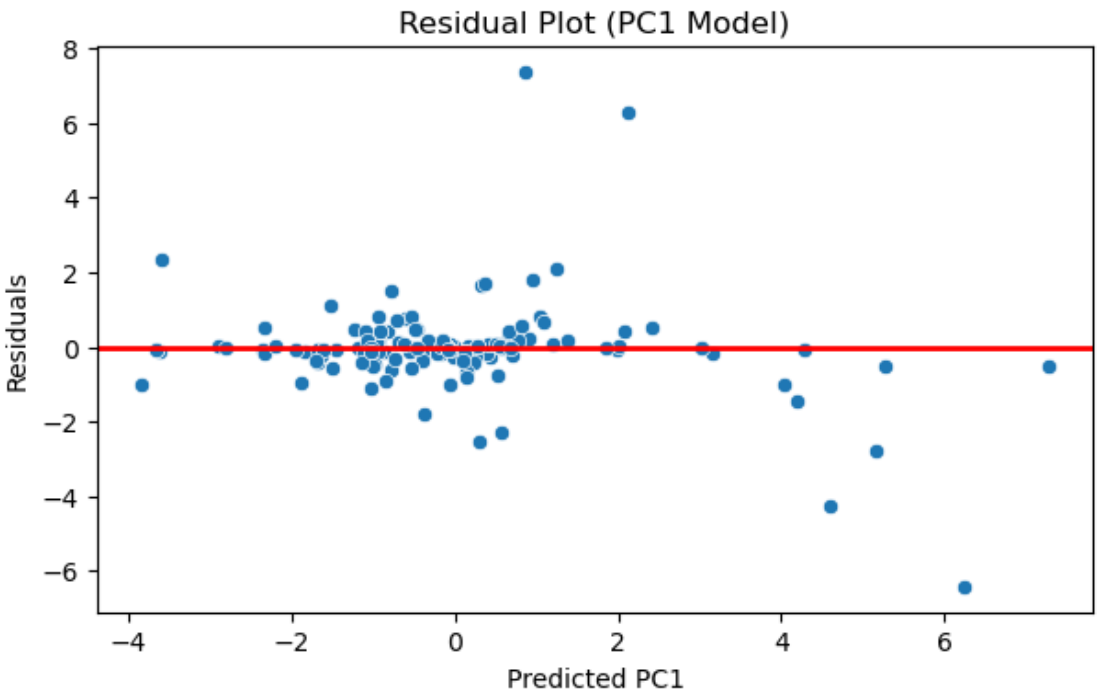


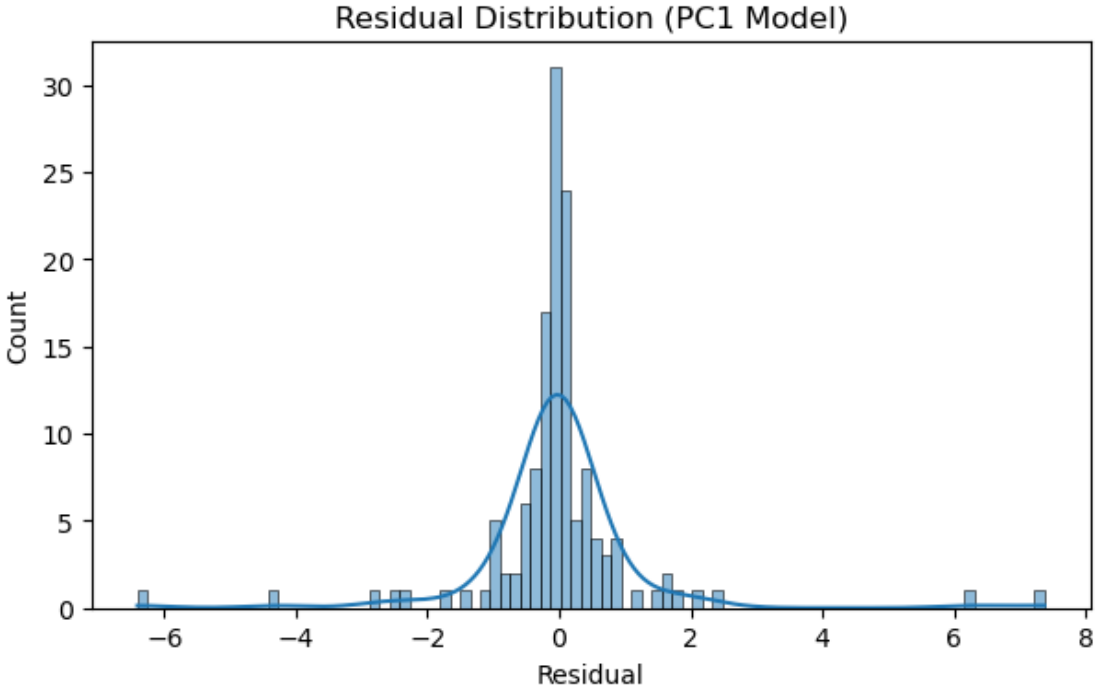
[34]:
```
# Overfitting check for Bucket Score model
train_preds = rf.predict(x_train_imp_df)
r2_train = r2_score(y_train, train_preds)
r2_test = r2_score(y_test, preds)

print(f"Train R²: {r2_train:.4f}")
print(f"Test  R²: {r2_test:.4f}")
```

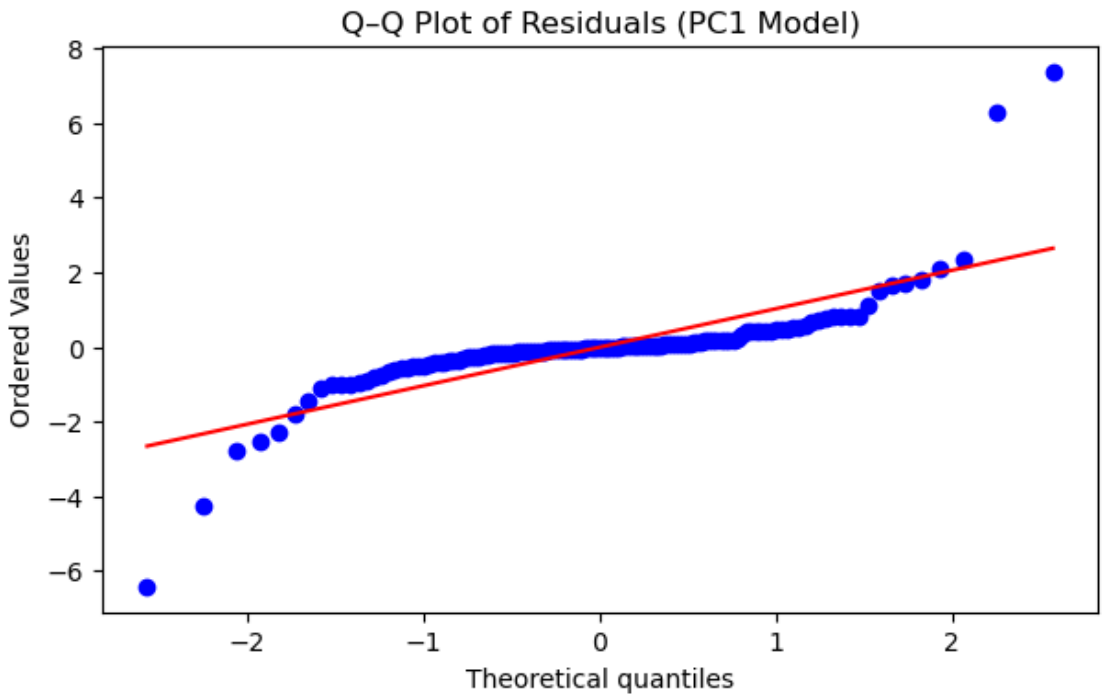
```
Train R²: 0.9718
Test  R²: 0.8545
```

[35]:
```
# ============================
# Residual Diagnostics for PC1
# ============================

# Residuals
residuals_2 = y_2_test - preds_2

# 1. Residual scatter plot
plt.figure(figsize=(7,4))
```

```
sns.scatterplot(x=preds_2, y=residuals_2)
plt.axhline(0, color='red', linewidth=2)
plt.xlabel("Predicted PC1")
plt.ylabel("Residuals")
plt.title("Residual Plot (PC1 Model)")
plt.show()

# 2. Residual histogram
plt.figure(figsize=(7,4))
sns.histplot(residuals_2, kde=True)
plt.title("Residual Distribution (PC1 Model)")
plt.xlabel("Residual")
plt.show()

# 3. Q-Q plot
plt.figure(figsize=(7,4))
stats.probplot(residuals_2, dist="norm", plot=plt)
plt.title("Q-Q Plot of Residuals (PC1 Model)")
plt.show()

# 4. Train vs Test R² (overfitting check)
train_preds_2 = rf_2.predict(x_2_train_imp)
r2_train_2 = r2_score(y_2_train, train_preds_2)
r2_test_2 = r2_score(y_2_test, preds_2)

print(f"PC1 Model - Train R²: {r2_train_2:.4f}")
print(f"PC1 Model - Test  R²: {r2_test_2:.4f}")
```


Residual Plot (PC1 Model)
Residuals
Predicted PC1
8
6
4
2
0
−2
−4
−6
−4
−2
0
2
4
6

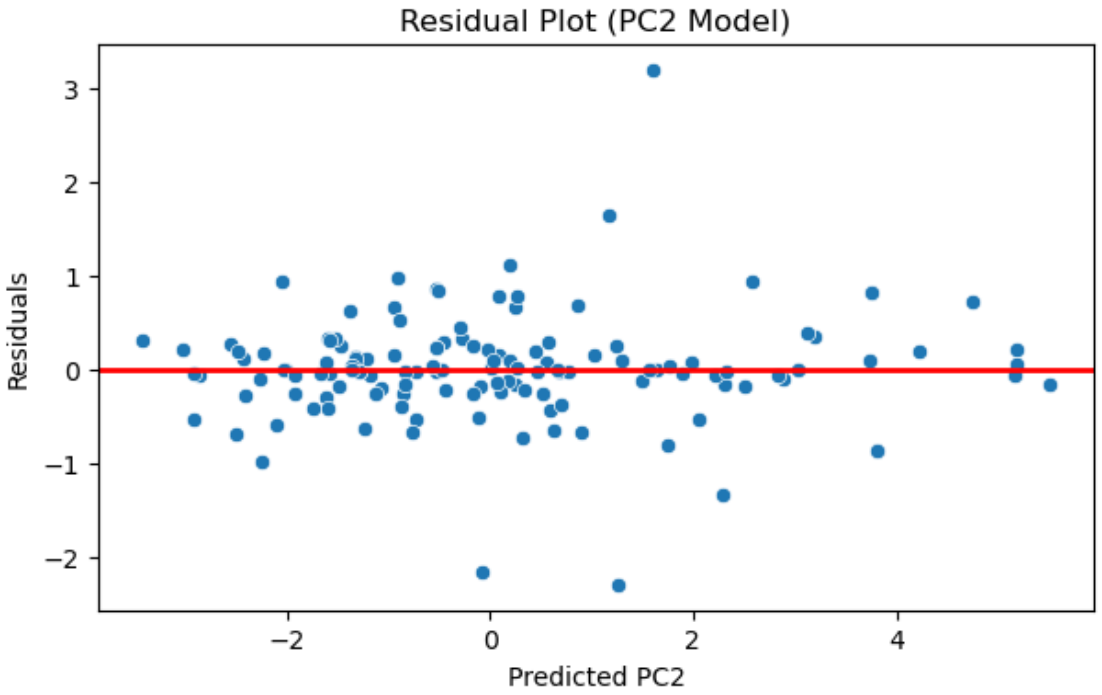

Residual Distribution (PC1 Model)
Count
Residual
30
25
20
15
10
5
0
−6
−4
−2
0
2
4
6
8

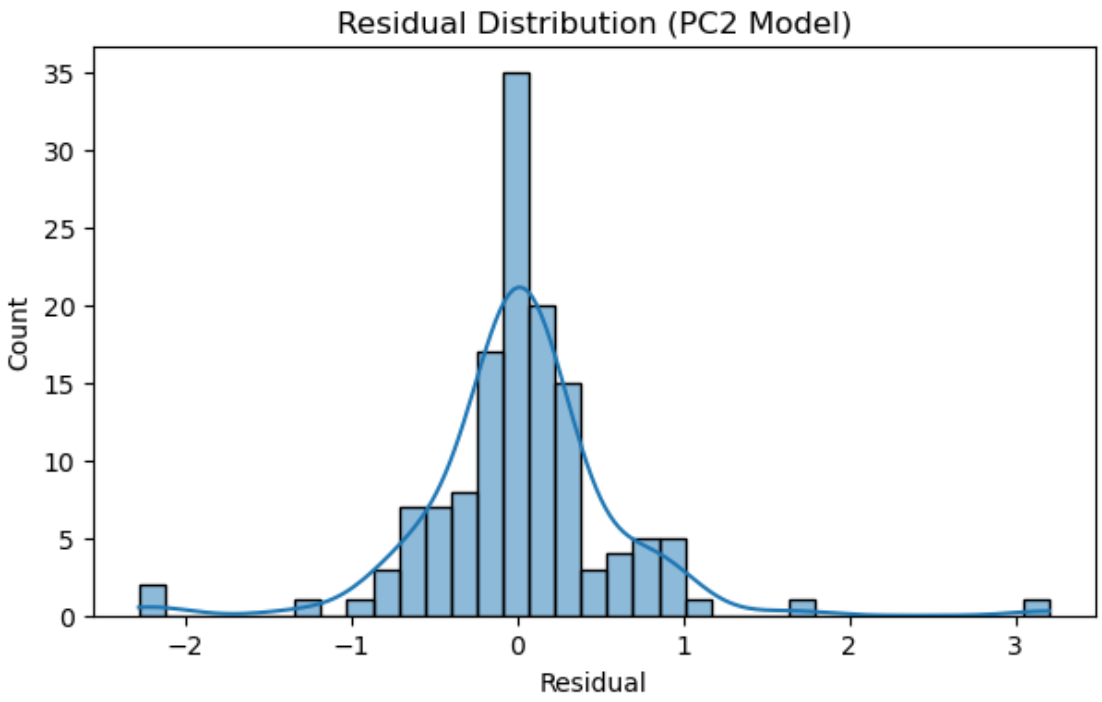


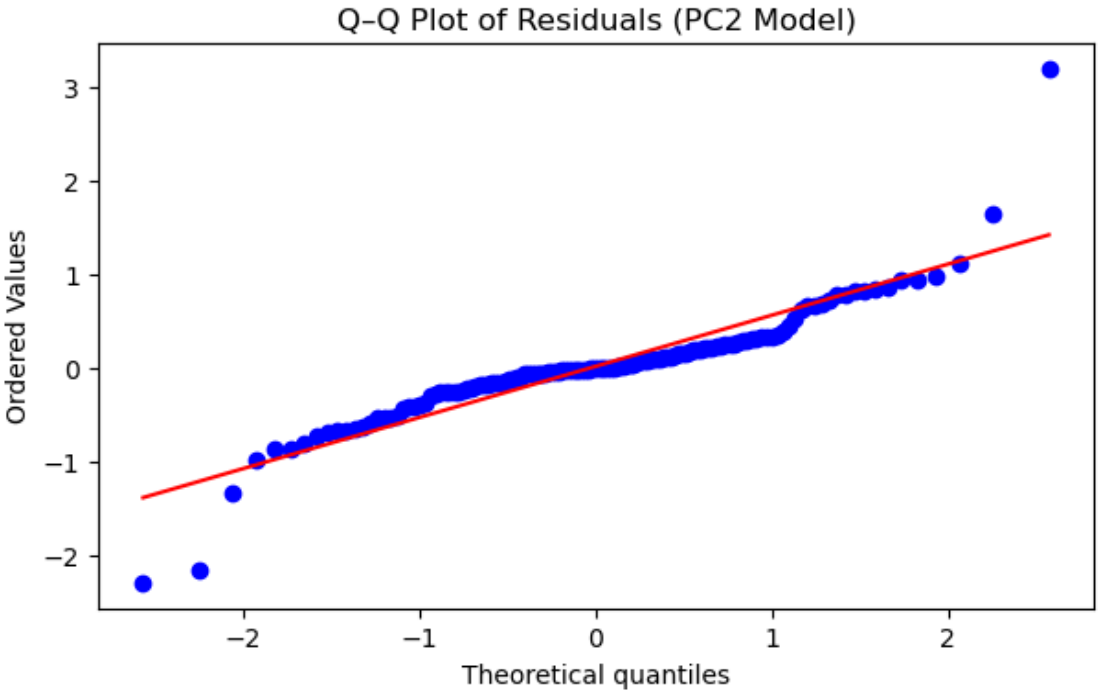

```
PC1 Model - Train R²: 0.9845
PC1 Model - Test  R²: 0.5992
```

[36]:
```python
# ============================
# Residual Diagnostics for PC2
# ============================

# Residuals
residuals_3 = y_3_test - preds_3

# 1. Residual scatter plot
plt.figure(figsize=(7,4))
sns.scatterplot(x=preds_3, y=residuals_3)
plt.axhline(0, color='red', linewidth=2)
plt.xlabel("Predicted PC2")
plt.ylabel("Residuals")
plt.title("Residual Plot (PC2 Model)")
plt.show()

# 2. Residual histogram
plt.figure(figsize=(7,4))
sns.histplot(residuals_3, kde=True)
plt.title("Residual Distribution (PC2 Model)")
plt.xlabel("Residual")
```

```
plt.show()

# 3. Q-Q plot
plt.figure(figsize=(7,4))
stats.probplot(residuals_3, dist="norm", plot=plt)
plt.title("Q-Q Plot of Residuals (PC2 Model)")
plt.show()

# 4. Train vs Test R² (overfitting check)
train_preds_3 = rf_3.predict(x_3_train_imp)
r2_train_3 = r2_score(y_3_train, train_preds_3)
r2_test_3 = r2_score(y_3_test, preds_3)

print(f"PC2 Model - Train R²: {r2_train_3:.4f}")
print(f"PC2 Model - Test  R²: {r2_test_3:.4f}")
```







PC2 Model - Train R^2: 0.9869

```
PC2 Model - Test  R²: 0.9229
```

[ ]: